

Task 1. 类、对象与构造方法

```
public class Test {
    public static void main(String[] args) {
        Person p = new Person("soyo", 16, 0);
    }
}

class Person {
    private String name;
    private int age;
    private int sex;

    public Person() {}

    public Person(String name, int age, int sex) {
        this.name = name;
        this.age = age;
        this.sex = sex;
    }

    public Person copy(Person p) {
        return new Person(p.name, p.age, p.sex);
    }

    private void eat() {
        System.out.println(name + "正在吃东西");
    }

    private void sleep() {

    }

    private void dadoudou() {

    }
}
```

1. 请说明你的构造方法是否用到了 this，如果用了，它在这里起什么作用

- 用了。表示对象的属性，区分成员变量和参数

2. 请用自己的话解释“类”和“对象”的关系

- 类是抽象的模版，对象是类的实例

Task 2. 访问修饰符、实例成员与静态成员

```
public class Test {
    public static void main(String[] args) {
        Person p = new Person("soyo", 16, 0);
    }
}
```

```

    }
}

class Person {
    public String name;
    public int age;
    public int sex;

    public static int count = 0;

    public Person() {}

    public Person(String name, int age, int sex) {
        this.name = name;
        this.age = age;
        this.sex = sex;
        count++;
    }

    public Person copy(Person p) {
        return new Person(p.name, p.age, p.sex);
    }

    public static void hello() {
        System.out.println("hello");
    }

    public void eat() {
        System.out.println(name + "正在吃东西");
    }

    public void sleep() {

    }

    public void dadoudou() {

    }
}

```

1. 说明 public、private、protected、默认访问权限分别代表什么

- public：完全对外公开，在任何包、任何类中都可以访问
- private：私有访问权限，只有类内部可以访问
- protected：同一包中可访问，不同包中只有子类可以访问
- 默认访问权限：同一个包中可以访问

2. 为什么静态方法可以直接通过类名调用，为什么实例方法通常需要先创建对象

- 静态方法属于类，类被加载到 JVM 时就有静态方法的信息了，不需要依赖任何对象
- 实例方法属于对象，通常需要操作对象属性，所以需要依赖对象

Task 3. 继承与代码复用

```
class Animal {
    private String name;
    private int id;

    public Animal(String myName, int myId) {
        this.name = myName;
        this.id = myId;
    }

    public String getName() {
        return name;
    }

    public int getId() {
        return id;
    }

    public void eat() {
        System.out.println(name + "正在吃");
    }

    public void sleep() {
        System.out.println(name + "正在睡");
    }

    public void introduction() {
        System.out.println("大家好! 我是" + id + "号" + name + "。");
    }
}

class Penguin extends Animal {
    public Penguin(String myName, int myId) {
        super(myName, myId);
    }

    @Override
    public void eat() {
        System.out.println(getName() + "一直在吃");
    }
}

class Mouse extends Animal {
    public Mouse(String myName, int myId) {
        super(myName, myId);
    }

    @Override
    public void eat() {
        System.out.println(getName() + "偷偷吃");
    }
}
```

```
}
```

1. 为什么 Java 不支持类的多继承？

- 多继承会使实现变得复杂，尤其是菱形继承，规则复杂且容易出错，所以为了保持语言的简单、安全，选择了单继承

Task 4. 多态与抽象设计

```
import java.util.ArrayList;
import java.util.List;

public class Task4 {
    public static void main(String[] args) {
        List<Graph> list = new ArrayList<>();
        list.add(new Circle("圆", 1));
        list.add(new Triangle("三角形", 3, 4, 5));
        list.add(new Rectangle("矩形", 1, 2));

        for (Graph g : list) {
            System.out.printf("%s: 面积=%.2f, 周长=%.2f\n",
                               g.getName(), g.area(), g.perimeter());
        }
    }
}

interface Graph {
    double area();

    double perimeter();

    String getName();
}

class Circle implements Graph {
    private String name;
    private double r;

    public Circle(String name, double r) {
        this.name = name;
        this.r = r;
    }

    @Override
    public double area() {
        return Math.PI * r * r;
    }

    @Override
    public double perimeter() {
        return 2 * Math.PI * r;
    }
}
```

```

    @Override
    public String getName() {
        return name;
    }
}

class Triangle implements Graph {
    private String name;
    private double a;
    private double b;
    private double c;

    public Triangle(String name, double a, double b, double c) {
        if (a + b <= c || a + c <= b || b + c <= a) {
            throw new IllegalArgumentException("不能构成三角形");
        }
        this.name = name;
        this.a = a;
        this.b = b;
        this.c = c;
    }

    @Override
    public double area() {
        double p = (a + b + c) / 2;
        return Math.sqrt(p * (p - a) * (p - b) * (p - c));
    }

    @Override
    public double perimeter() {
        return a + b + c;
    }

    @Override
    public String getName() {
        return name;
    }
}

class Rectangle implements Graph {
    private String name;
    private double a;
    private double b;

    public Rectangle(String name, double a, double b) {
        this.name = name;
        this.a = a;
        this.b = b;
    }

    @Override
    public double area() {

```

```

        return a * b;
    }

    @Override
    public double perimeter() {
        return 2 * (a + b);
    }

    @Override
    public String getName() {
        return name;
    }
}

```

Task 5. 封装与接口设计

```

class BankAccount {
    private final String accountNumber;
    private final String accountHolder;
    private double balance;
    private String password;

    public BankAccount(String accountNumber, String accountHolder,
                        double initialBalance, String password) {
        if (accountNumber == null || accountNumber.trim().isEmpty()) {
            throw new IllegalArgumentException("账号不能为空");
        }
        if (accountHolder == null || accountHolder.trim().isEmpty()) {
            throw new IllegalArgumentException("户名不能为空");
        }
        if (initialBalance < 0) {
            throw new IllegalArgumentException("初始余额不能为负数");
        }
        if (password == null || password.isEmpty()) {
            throw new IllegalArgumentException("密码不能为空");
        }

        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
        this.password = password;
    }

    public void deposit(double amount) {
        if (!validateAmount(amount)) {
            throw new IllegalArgumentException("存款金额必须大于 0");
        }
        balance += amount;
    }

    public boolean withdraw(double amount, String inputPassword) {

```

```

        if (!validatePassword(inputPassword)) {
            return false;
        }

        if (!validateAmount(amount)) {
            return false;
        }

        if (amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

```

```

    public boolean transfer(BankAccount recipient, double amount, String inputPassword) {

        if (recipient == null || recipient == this) {
            return false;
        }

        if (!validatePassword(inputPassword)) {
            return false;
        }

        if (!validateAmount(amount)) {
            return false;
        }

        if (amount > balance) {
            return false;
        }

        this.balance -= amount;
        recipient.balance += amount;
        return true;
    }

```

```

    public double getBalance() {
        return balance;
    }

```

```

    public String getAccountInfo() {
        String maskedAccountNumber;
        if (accountNumber.length() > 4) {
            maskedAccountNumber = "*****" + accountNumber.substring(accountNumber.length() -
4);
        } else {
            maskedAccountNumber = "*****";
        }
    }

```

```

    // 不返回密码
    return "账户号: " + maskedAccountNumber
        + ", 户名: " + accountHolder

```

```
        + ", 余额: " + balance;
    }

    private boolean validatePassword(String inputPassword) {
        return inputPassword != null && inputPassword.equals(password);
    }

    private boolean validateAmount(double amount) {
        return amount > 0
            && !Double.isNaN(amount)
            && !Double.isInfinite(amount);
    }
}
```